

LLM and Search

From Lexical Retrieval to Agentic Information Retrieval

A Master's/Doctoral-Level Technical Report for Tsinghua University

Prepared for: Advanced Research Seminar

Prepared by: Mauricio Montes

Date: April 12, 2026

Purpose: to develop expert-level understanding of how large language models (LLMs) transform information retrieval (IR) across search, ranking, grounding, generation, and agentic action.

□

Abstract

The convergence of large language models (LLMs) and information retrieval (IR) has shifted the field from static ranking toward systems that can rewrite queries, retrieve evidence, rerank candidates, synthesize grounded answers, and increasingly act as autonomous search agents. This report consolidates the field into three dominant paradigms: LLMs for Search, Search for LLMs, and LLMs as Search. The first paradigm uses LLMs to improve classical or neural retrieval pipelines; the second uses retrieval to ground generation and reduce hallucination; the third attempts to internalize retrieval inside the model through generative retrieval. Across these paradigms, the central design trade-offs are not limited to relevance and recall. They also include factuality, latency, updateability, interpretability, robustness, and operational cost. The report explains why BM25 remains essential, why dense retrieval changed the geometry of relevance, why RAG became the default grounding architecture, why long-context LLMs do not eliminate the need for retrieval, and why the next frontier is likely to be agentic and tool-augmented IR (Robertson & Zaragoza, 2009; Karpukhin et al., 2020; Zhu et al., 2023; Lewis et al., 2020; Tay et al., 2022; Asai et al., 2024; Zhang et al., 2024; Li et al., 2024).

How to read this report

Read the report in sequence if your goal is mastery. The early sections build the conceptual foundation, the middle sections explain the major architectures, and the later sections focus on trade-offs, failure modes, and research frontiers. The final section is designed as a seminar preparation guide and a self-test for expert-level understanding.

1. Why LLMs changed search

For decades, search was treated as a ranking problem: given a query and a corpus, return the most relevant documents. Classical IR solved this with sparse lexical models such as TF-IDF and BM25, which rely on an inverted index, term weighting, and document-length normalization. This architecture remains foundational because it is fast, scalable, interpretable, and excellent for exact matching, named entities, identifiers, and other low-ambiguity queries (Robertson & Zaragoza, 2009).

The modern shift began when neural retrieval showed that relevance can be modeled in a latent semantic space rather than only through word overlap. Dense Passage Retrieval (DPR) demonstrated that dual encoders can outperform a strong BM25 baseline on open-domain question answering by learning query and passage representations from supervision (Karpukhin et al., 2020). LLMs expanded this further: they do not merely encode language, but can rewrite it, reason over it, generate it, and critique their own outputs. Search therefore evolved from item retrieval into a broader information pipeline that may include planning, evidence selection, synthesis, and action (Zhu et al., 2023; Zhang et al., 2024).

The deepest conceptual change is that the objective is no longer just relevance@k. In LLM-based search, systems are increasingly optimized for utility under constraints: factual correctness, answer completeness, freshness of evidence, latency, and cost. In other words, the system is judged not only by what it retrieves, but by how well it helps the user complete a task.

2. Foundational papers you must own conceptually

The following six works form a compact reading spine for the field. Together they explain the transition from classical retrieval to retrieval-augmented generation, differentiable search, self-reflective grounding,

and agentic information access.

Paper	What it introduced	Why it matters	How to use it in your mental model
Zhu et al. (2023)	A field survey of how LLMs intervene in query rewriting, retrieval, reranking, reading, and search agents.	It is the map of the entire landscape.	Use it as the taxonomy layer: what can an LLM do inside IR?
Lewis et al. (2020)	Retrieval-Augmented Generation (RAG) with parametric and non-parametric memory.	It established retrieval as grounding for generation.	Use it as the default answer to: how do we give an LLM fresh evidence?
Tay et al. (2022)	Differentiable Search Index (DSI), where the model generates document identifiers directly.	It is the radical 'LLMs as Search' baseline.	Use it to understand end-to-end retrieval without a traditional external index.
Asai et al. (2024)	Self-RAG with reflection tokens and adaptive retrieval decisions.	It adds self-critique and retrieval control.	Use it to reason about when retrieval should happen, not just whether it can happen.
Zhang et al. (2024)	Agentic Information Retrieval as a loop of observation, reasoning, and action.	It reframes search as task-oriented information state transition.	Use it when search becomes multi-step, interactive, and tool-driven.
Li et al. (2024)	A direct comparison between RAG and long-context LLMs, plus the Self-Route idea.	It clarifies the key strategic trade-off in current systems.	Use it to decide when retrieval is better than just reading more context.

3. Evolution of the field

3.1 Lexical retrieval

Lexical retrieval treats text as a collection of terms and scores documents according to their statistical relation to the query. BM25 is the canonical model of this family. Its practical success comes from a simple but powerful idea: relevant terms should matter more when they appear frequently, but with diminishing returns, and document length should be normalized so that long documents are not unfairly advantaged (Robertson & Zaragoza, 2009).

The limitation of lexical retrieval is not that it is weak everywhere; rather, it uses a shallow notion of relevance. When users express intent indirectly, paraphrastically, or with incomplete terminology, lexical matching suffers from vocabulary mismatch. BM25 therefore works best when exact strings matter, while semantic understanding is still outside its design scope.

BM25 can be written as a weighted sum over query terms. In simplified form, the score grows with term frequency, saturates with repetition, and normalizes by document length. The notation is less important than the intuition: frequency saturates, long documents are normalized, and rare terms receive larger weight.

3.2 Neural dense retrieval

Dense retrieval addresses vocabulary mismatch by learning embeddings for queries and documents. Instead of matching words explicitly, the model learns to place semantically related texts near one another in a latent vector space. DPR is one of the most influential examples of this direction, showing that a dual-encoder architecture can retrieve relevant passages effectively for open-domain QA (Karpukhin et al., 2020).

The neural retrieval pipeline typically separates the problem into two stages. A first-stage retriever produces a candidate set quickly, and a second-stage reranker reorders those candidates with a more expensive but more accurate model. This is the architecture that made modern search systems practical: the first stage preserves scale, while the second stage recovers semantic precision.

3.3 The LLM era

LLMs changed IR because they are simultaneously language models, reasoning engines, and tool controllers. The survey by Zhu et al. (2023) organizes the field around query rewriting, retrievers, rerankers, readers, and search agents. The conceptual shift is significant: search is no longer a fixed retrieval routine, but a compositional process that may include planning, evidence selection, synthesis, and self-critique.

This is why LLM-based search should not be reduced to 'chat with search.' The deeper transformation is architectural: the system can now decide what to ask, what to retrieve, how much context to keep, whether the retrieved evidence is good enough, and when to answer or continue searching.

4. A conceptual map of LLM + search systems

To understand the literature cleanly, it helps to separate four design questions. First, where is the knowledge stored: in a sparse index, in dense embeddings, in the model parameters, or in an external tool? Second, when is retrieval performed: before generation, during generation, after generation, or only when needed? Third, what is the system optimizing: ranking accuracy, factual grounding, response quality, or task completion? Fourth, how is the result verified: by retrieval metrics, faithfulness checks, citations, or human evaluation?

These questions explain why the field now spans very different systems that are all called 'search.' A BM25 pipeline, a dense retriever, a RAG assistant, a generative retriever, and a multi-agent research system all address different parts of the same information problem.

5. Taxonomy of contemporary architectures

A useful taxonomy divides the field into three major paradigms, with agentic IR as an emerging extension. The three core paradigms are LLMs for Search, Search for LLMs, and LLMs as Search. This classification is more than a naming convention. It tells us whether the LLM is acting as a helper to retrieval, a consumer of retrieval, or a replacement for the traditional index.

5.1 LLMs for Search

In this category, LLMs improve an existing search system without replacing it. Common uses include query rewriting, query expansion, pseudo-document generation, reranking, and tool selection. Query rewriting is especially important in conversational search because users often omit context that the retrieval system needs. Ma et al. (2023) formalized a rewrite-retrieve-read framework to narrow the gap between user wording and corpus wording.

5.2 Search for LLMs

Retrieval-Augmented Generation (RAG) is the best-known example of search supporting generation. The idea is simple: retrieve external evidence at inference time, then condition the LLM on that evidence so the answer is grounded in up-to-date, domain-specific, or private content (Lewis et al., 2020). RAG emerged as a practical response to the limitations of parametric memory: hallucination, stale knowledge, and non-transparent reasoning.

5.3 LLMs as Search

The most radical line of work attempts to internalize retrieval inside the model. Generative Information Retrieval (GenIR) and differentiable search approaches map queries directly to document identifiers or other retrieval targets, reducing or eliminating the need for a traditional external index (Li et al., 2024). In this view, the model itself becomes part of the search infrastructure.

5.4 Agentic IR and tool use

Agentic IR extends the previous paradigms by framing information access as a sequence of observations, actions, and state updates rather than a single retrieval event. Zhang et al. (2024) argue that the definition of 'information' is shifting from static items to context-dependent information states. In this view, the system is not only searching for documents, but steering the user toward a task-relevant state.

6. Method-level deep dive

6.1 Sparse retrieval: why BM25 still matters

BM25 remains a first-class baseline in modern retrieval systems for a reason. It is highly optimized, easy to index, easy to debug, and surprisingly competitive on exact-match and domain-specific queries. In legal search, product search, medical coding, error-message lookup, and other identifier-heavy domains, lexical signals can outperform or complement semantic embeddings because the exact surface form contains critical meaning (Robertson & Zaragoza, 2009; Zhu et al., 2023).

A mature expert should understand BM25 not as an outdated relic, but as a strong sparse prior. It remains the best first-stage filter in many hybrid systems because it cheaply captures rare tokens and named entities. Dense models are excellent at semantic recall, but sparse methods are still superior when the query demands exact lexical evidence.

6.2 Dense retrieval: semantic geometry

Dense retrieval replaces token overlap with learned geometry. The basic objective is to map queries and passages into a shared vector space so that semantically related items are close under dot product or cosine similarity. The power of this approach is that it can bridge paraphrase, synonymy, and incomplete phrasing. Its weakness is that it depends on training data, embedding quality, and a good indexing backend (Karpukhin et al., 2020; Zhao et al., 2024).

The expert-level point here is that dense retrieval is not merely a better encoder. It changes what a retrieval system is optimizing. Instead of matching symbolic evidence, the model learns a relevance manifold. That manifold can be very effective, but it can also collapse distinctions that matter for exact search.

6.3 Query rewriting and query expansion

Query rewriting addresses one of the most persistent problems in search: the user's words often underspecify the user's intent. In conversational search, this is especially severe because pronouns, ellipsis, and context dependence create a mismatch between the utterance and the standalone query required by the retriever. LLMs can rewrite the query into a more explicit form, add missing context, or generate alternative phrasings that improve recall (Ma et al., 2023).

HyDE is a more aggressive form of query expansion. Rather than rewriting the query into another query, it asks the LLM to hallucinate a plausible document and then uses that document as the retrieval anchor. This is powerful because it moves the query into document space, but it also creates a risk: if the hypothetical document is too speculative, the retriever may drift. Good HyDE systems therefore rely on the embedding bottleneck to suppress hallucinated details (Gao et al., 2022).

6.4 Reranking: from candidate sets to decision sets

Reranking is where LLMs often deliver the highest practical value per query. A cheap retriever first produces a top-k candidate set; then a stronger model examines those candidates jointly and orders them by relevance. Sun et al. (2023) showed that LLMs can be surprisingly strong rerankers when prompted carefully. The listwise view is important because relevance is relational: the best document is not only good by itself, but better than the others in the candidate set.

More recent reranking work seeks to reduce the cost of using full generative decoding. Attention-based reranking and in-context reranking exploit internal signals from the transformer instead of asking the model to generate an explicit ranking string. This matters because latency, token budget, and throughput can be decisive in production search systems (Chen et al., 2024).

6.5 Retrieval-augmented generation

RAG is now the default architecture when external knowledge must be grounded into an LLM. In its simplest form, the model retrieves evidence, concatenates it into the prompt, and generates an answer conditioned on that evidence (Lewis et al., 2020). In modern systems, the pipeline is usually more elaborate: query rewriting, retrieval, context filtering, context compression, answer generation, and sometimes citation generation or verification.

The RAG literature has converged on a key insight: the quality of the answer depends on both retrieval quality and generation quality. If the retriever misses the right evidence, the generator cannot recover. If the retriever is good but the generator ignores the evidence it receives, the system still fails. This is why RAG evaluation cannot rely on retrieval metrics alone (Wu et al., 2024; Zhao et al., 2024).

6.6 Self-RAG and adaptive grounding

Self-RAG is especially important because it introduces self-reflection into the pipeline. The model learns to decide whether retrieval is needed, whether retrieved evidence is relevant, and whether the answer is supported. That makes the system more adaptive than fixed-top-k RAG and moves it toward a real deliberative policy (Asai et al., 2024).

CoRAG pushes this further by allowing retrieval to unfold as a chain. This is useful for complex questions because the information need often changes after the first evidence chunk is read. Retrieval then becomes iterative rather than one-shot (Wang et al., 2025).

6.7 Long-context LLMs

One of the most debated questions in the field is whether retrieval is still necessary when the model can handle very long contexts. The answer from the literature is nuanced. Long-context LLMs can be very strong when the relevant evidence is self-contained and the prompt budget is sufficient, but they are expensive, and their ability to use the middle of a long input remains imperfect (Liu et al., 2023; Li et al., 2024).

A useful way to think about the trade-off is this: long context increases what the model can see, but retrieval controls what the model must pay attention to. RAG is often cheaper because it compresses the relevant evidence before generation. Long-context is attractive when you want broad local reasoning over a bounded corpus, but it is not a universal replacement for retrieval.

6.8 Generative retrieval

Generative retrieval is the most ambitious attempt to unify retrieval and generation. Instead of searching an external index, the model generates a document identifier or other retrieval target directly. The survey by Li et al. (2024) distinguishes generative document retrieval from reliable response generation. Both directions replace discrete index lookup with learned generation over retrieval targets.

The model must memorize or approximate a vast mapping from query space to document space, which creates scaling problems, update problems, and sensitivity to identifier design. DSI is the classical reference point here, and DGI pushes the frontier further by attacking optimization blockage and geometric instability in end-to-end generative retrieval (Tay et al., 2022; Wang et al., 2026).

7. Comparative analysis: what actually differs?

The real differences between systems are easiest to see when you compare them across the dimensions that matter in practice: latency, scale, cost, factuality, updateability, and interpretability. The table below is a synthesis of the dominant trade-offs reported in recent surveys and representative papers.

Paradigm	Latency	Scale	Factuality	Updateability	Best use case
BM25 + rerank	Low to medium	Excellent	High for exact strings	Very high	Exact-match, high-throughput search
Dense retrieval	Low to medium	High	High semantic recall	High	Paraphrase-heavy semantic search
RAG	Medium	High	High when retrieval is correct	High	Grounded QA and enterprise assistants
Long-context LLM	High	Limited by window	High when evidence fits	Medium	Bounded corpora and long-form reasoning
Generative retrieval	Medium	Medium to low	Variable	Low to medium	Unified end-to-end retrieval research

The practical takeaway is not that one paradigm dominates all others. BM25 remains excellent when exact matching matters. Dense retrieval is strongest when semantics dominate. RAG is the safest default when grounded answers are required. Long-context LLMs are attractive when the corpus is bounded and the budget is generous. Generative retrieval is the most radical architecture, but not yet the most operationally mature (Lewis et al., 2020; Tay et al., 2022; Li et al., 2024; Zhang et al., 2024).

8. Core challenges and failure modes

8.1 Hallucination and unsupported generation

Hallucination is not solved by retrieval alone. A RAG system can still fail if it retrieves the wrong evidence or if the generator ignores the evidence it receives. The deeper problem is faithfulness: the output must be supported by the retrieved material, not just linguistically plausible. This is why self-critique, verification, and citation-aware generation have become central topics (Asai et al., 2024; Zhao et al., 2024).

8.2 Long-context degradation

Increasing context length does not guarantee better use of context. A persistent finding is that language models may underutilize information placed in the middle of long inputs, a phenomenon often called 'lost in the middle' (Liu et al., 2023). That makes naive long-context prompting unreliable as a universal alternative to retrieval.

8.3 Staleness and updateability

Parametric memory is static relative to the training cutoff. That is a major problem in domains where facts change quickly. RAG addresses this by attaching fresh external evidence to the model at inference time. Generative retrieval, by contrast, makes updates harder because some of the knowledge is entangled in the model itself (Lewis et al., 2020; Li et al., 2024).

8.4 Bias, robustness, and adversarial behavior

Search systems inherit bias at two levels: from the model and from the corpus. A model may rank or summarize evidence in ways that reflect pretraining bias, while the corpus may overrepresent certain viewpoints or categories. Tool selection and retrieval are also vulnerable to prompt injection, keyword stuffing, and other adversarial strategies. Grounding does not equal neutrality.

8.5 Evaluation is multi-objective

Traditional IR metrics such as Recall@k, MRR, and nDCG remain necessary, but they are insufficient once the system generates answers. The evaluation must also include faithfulness, citation accuracy, completeness, cost, and task success. This is especially true for RAG and agentic systems, where the output may be a multi-step process rather than a single answer (Wu et al., 2024; Zhang et al., 2024).

9. Future directions

9.1 Routing and hybrid architectures

The most plausible future is not a single dominant architecture, but a hybrid ecosystem. BM25 will remain the sparse backbone. Dense retrieval will continue to power semantic candidate generation. LLM rerankers will become more efficient. RAG will remain the grounding default for factual and dynamic tasks. Long-context models will be chosen when the evidence can reasonably fit in the prompt. Routing methods will decide which of these tools to use per query (Li et al., 2024).

9.2 Agentic IR

Agentic IR is the deeper frontier. Rather than retrieving a list of documents once, the system observes user needs, reasons about sufficiency, calls tools, updates state, and iterates until the information goal is reached (Zhang et al., 2024). This matters for coding assistants, scientific discovery, legal search, and decision support, where the objective is not document access but task completion.

9.3 Retrieval chains and stepwise reasoning

Chain-based retrieval suggests that retrieval should be modeled as a sequence rather than a single event. That is an important shift because the information need often evolves during reasoning. A strong system should be able to re-query, reformulate, and retrieve again when early evidence is insufficient (Wang et al., 2025).

9.4 More structured retrieval

Another direction is to move beyond flat chunking toward richer structure: hierarchies, graphs, multimodal documents, and retrieval of rationales instead of only passages. In that future, search is less about finding a single relevant chunk and more about assembling the right evidence structure for reasoning.

9.5 Differentiable and geometric retrieval

DGI is representative of a broader trend: treating retrieval as a differentiable object that can be optimized end-to-end while preserving geometric stability (Tay et al., 2022; Wang et al., 2026). If this direction matures, it may reduce the gap between index construction and model training.

10. How to become expert-level in the topic

To become genuinely expert in LLM and search, it helps to learn the field through five lenses rather than through a single paper list. First, learn retrieval geometry: how sparse and dense systems represent relevance. Second, learn grounding: how retrieval supports trustworthy generation. Third, learn orchestration: how query rewriting, reranking, and tool use are composed into pipelines. Fourth, learn evaluation: how to measure both retrieval and answer quality. Fifth, learn agency: how systems plan, act, reflect, and adapt.

A strong reading order is BM25 and the probabilistic relevance framework, then dense retrieval and DPR, then RAG and its modern surveys, then Self-RAG and long-context comparisons, and finally agentic IR and generative retrieval. If you can explain why each step was necessary and what it fixed, you are already operating at a graduate-research level.

The most useful expert habit is to ask, for every new paper: what part of the pipeline does it change, what failure mode does it target, what is the new trade-off, and what does it leave unresolved?

11. Seminar Q&A preparation

Q: Why does BM25 still matter in the LLM era?

A: Because exact lexical matching is still unmatched for identifiers, rare terms, and highly precise queries; BM25 is also cheap, interpretable, and a strong first-stage prior.

Q: Why not replace RAG with a long-context LLM?

A: Because long-context inference is expensive, not always reliable in the middle of the window, and often less updateable than retrieval-based systems.

Q: What is the main advantage of Self-RAG?

A: It allows the model to decide when to retrieve and how to critique evidence, rather than forcing fixed retrieval at every step.

Q: What is the main weakness of generative retrieval?

A: It is hard to update, can be sensitive to identifier design, and struggles to scale cleanly to large, dynamic corpora.

Q: What is the deepest conceptual change introduced by agentic IR?

A: The goal shifts from returning documents to helping the user reach a desired information state through planning, tool use, and iteration.

12. Why these six papers are the core reading map

The six works highlighted above are intentionally complementary. Zhu et al. (2023) gives the taxonomy of the entire field; Lewis et al. (2020) provides the grounding architecture that made RAG the default; Tay et al. (2022) shows the most radical internalization of search; Asai et al. (2024) adds adaptive control and self-reflection; Zhang et al. (2024) defines the agentic frontier; and Li et al. (2024) frames the current strategic debate between retrieval and long context. Together, they explain why modern search is no longer only about finding text, but about selecting, grounding, synthesizing, and acting on information.

References

- Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In Proceedings of the International Conference on Learning Representations (ICLR 2024). <https://arxiv.org/abs/2310.11511>
- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). <https://arxiv.org/abs/2004.04906>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems, 33. <https://arxiv.org/abs/2005.11401>
- Li, X., Jin, J., Zhou, Y., Zhang, Y., Zhang, P., Zhu, Y., & Dou, Z. (2024). From matching to generation: A survey on generative information retrieval. arXiv. <https://arxiv.org/abs/2404.14851>
- Li, Z., Li, C., Zhang, M., Mei, Q., & Bendersky, M. (2024). Retrieval augmented generation or long-context LLMs? A comprehensive study and hybrid approach. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track. <https://arxiv.org/abs/2407.16833>
- Tay, Y., Tran, V. Q., Dehghani, M., Ni, J., Bahri, D., Mehta, H., Qin, Z., Hui, K., Zhao, Z., Gupta, J., Schuster, T., Cohen, W. W., & Metzler, D. (2022). Transformer memory as a differentiable search index. In Advances in Neural Information Processing Systems, 35. <https://arxiv.org/abs/2202.06991>
- Wang, X., Chen, Y., Zhang, B., Zhao, J., Wei, C., Xu, C., Guan, Z., Zhao, W., Zhang, W., & Zeng, X. (2026). Differentiable geometric indexing for end-to-end generative retrieval. arXiv. <https://arxiv.org/abs/2603.10409>
- Wang, L., Chen, H., Yang, N., Huang, X., Dou, Z., & Wei, F. (2025). Chain-of-retrieval augmented generation. arXiv. <https://arxiv.org/abs/2501.14342>
- Wu, S., Xiong, Y., Cui, Y., Wu, H., Chen, C., Yuan, Y., Huang, L., Liu, X., Kuo, T.-W., Guan, N., & Xue, C. J. (2024). Retrieval-augmented generation for natural language processing: A survey. arXiv. <https://arxiv.org/abs/2407.13193>
- Yao, S., Zhao, J., Yu, D., Shafraan, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In Proceedings of the International Conference on Learning Representations (ICLR 2023). <https://arxiv.org/abs/2210.03629>
- Zhang, W., Liao, J., Li, N., Du, K., & Lin, J. (2024). Agentic information retrieval. arXiv. <https://arxiv.org/abs/2410.09713>
- Zhao, S., Yang, Y., Wang, Z., He, Z., Qiu, L. K., & Qiu, L. (2024). Retrieval augmented generation (RAG) and beyond: A comprehensive survey on how to make your LLMs use external data more wisely. arXiv. <https://arxiv.org/abs/2409.14924>
- Zhu, Y., Yuan, H., Wang, S., Liu, J., Liu, W., Deng, C., Chen, H., Liu, Z., Dou, Z., & Wen, J.-R. (2023). Large language models for information retrieval: A survey. arXiv. <https://arxiv.org/abs/2308.07107>
- Zheng, Y., Li, P., Liu, W., Liu, Y., Luan, J., & Wang, B. (2024). Adaptive and hierarchy-aware reranking for tool retrieval. arXiv. <https://arxiv.org/abs/2403.06551>
- Ma, X., Gong, Y., He, P., Zhao, H., & Duan, N. (2023). Query rewriting for retrieval-augmented large language models. arXiv. <https://arxiv.org/abs/2305.14283>
- Gao, L., Ma, X., Lin, J., & Callan, J. (2022). Precise zero-shot dense retrieval without relevance labels. arXiv. <https://arxiv.org/abs/2212.10496>